

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
«Конструювання програмного забезпечення»

Лабораторна робота № 5
Варіант 1

Виконав:	Приймаченко Артем Андрійович	Перевірив:	Шабатін Павло Євгенійович
Група	ІПЗ-34-10	Дата перевірки	
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Варіант завдання: Симулятор системи залежностей (Dependency resolver).

Хід роботи:

Створимо Dockerfile. Беремо docker-образ із .NET SDK (щоб мати можливість компілювати проект), створюємо робочу директорію в контейнері:

```
1 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
2 WORKDIR /App
```

Копіюємо весь проект із поточної папки в контейнер:

```
4 # Copy everything
5 COPY . ./
```

Встановлюємо всі залежності та збираємо готовий до запуску застосунок:

```
6 # Restore as distinct layers
7 RUN dotnet restore
8 # Build and publish a release
9 RUN dotnet publish -c Release -o out
```

Далі для оптимізації використовується новий docker-образ, але вже тільки з .NET Runtime без SDK. Встановлюємо робочу директорію, і копіюємо всі файли з папки out в поточну директорію. Також вказуємо команду запуску контейнера:

```
11 # Build runtime image
12 FROM mcr.microsoft.com/dotnet/runtime:8.0
13 WORKDIR /App
14 COPY --from=build /App/out .
15 ENTRYPOINT ["dotnet", "lab1.dll"]
```

Також створимо .dockerignore:

```
1 bin/
2 obj/
3 out/
4
5 TestResults/
6 CoverageReport/
7
8 .git/
9
10 .vs/
11 *.user
```

Створимо файл конфігурації ci.yml, що буде запускатись щоразу при push або pull request в основну гілку main:

```
1   name: CI
2
3   on:
4     push:
5       branches: [ main ]
6     pull_request:
7       branches: [ main ]
```

Спочатку завантажимо код репозиторію:

```
9   jobs:
10     build-and-test:
11       runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout code
15         uses: actions/checkout@v4
```

Встановимо .NET:

```
16
17     - name: Setup .NET
18       uses: actions/setup-dotnet@v4
19       with:
20         dotnet-version: 8.0.x
```

Далі встановимо всі залежності проекту:

```
22     - name: Restore Dependencies
23       run: dotnet restore
```

Запускаємо перевірку літером:

```
25     - name: Run Linter Check
26       run: dotnet format analyzers --verify-no-changes
```

Компілюємо проект:

```
28     - name: Build Project
29       run: dotnet build -c Release --no-restore
```

І запускаємо тести:

```
31     - name: Run Tests
32       run: dotnet test -c Release --no-build
```

Створимо cd.yml, що буде виконуватись при пуші тегів, які відповідають шаблону “v.x.x”:

```

1     name: CD
2
3     on:
4         push:
5             tags:
6                 - 'v*.*.*'

```

Далі повторюємо етапи попереднього файлу. Вантажимо репозиторій, встановлюємо .NET та залежності проекту, запускаємо перевірку лінтером, компілюємо його та запускаємо тести:

```

8     jobs:
9         release:
10            runs-on: ubuntu-latest
11
12            steps:
13                - name: Checkout code
14                  uses: actions/checkout@v4
15
16                - name: Setup .NET
17                  uses: actions/setup-dotnet@v4
18                  with:
19                      dotnet-version: 8.0.x
20
21                - name: Restore Dependencies
22                  run: dotnet restore
23
24                - name: Run Linter Check
25                  run: dotnet format analyzers --verify-no-changes
26
27                - name: Build Project
28                  run: dotnet build -c Release --no-restore
29
30                - name: Run Tests
31                  run: dotnet test -c Release --no-build

```

Створимо артефакт (готову до запуску версію застосунку) у папці out:

```

33                - name: Publish Project
34                  run: dotnet publish -c Release -o out

```

Архівуємо папку out:

```

36                - name: Archive Artifact
37                  run: zip -r release.zip out

```

І використаємо GitHub Action, що створить Release та додасть до нього створений архів release.zip:

```
39     - name: Create Release
40       uses: softprops/action-gh-release@v2
41       with:
42         files: release.zip
```